

SPRINT 7

Resoldràs alguns problemes de la vida quotidiana aplicant les estructures de dades i control en Python.

Un client de l'empresa en la qual treballes demana una llista de programes molt senzills, però que li facilitarien molts processos. No obstant això, el departament de TU està molt complicat amb el temps, per la qual cosa et demanen que facis la programació.

Nivell 1

Exercici 1

Calculadora de l'índex de massa corporal:

- Escriu una **funció** que calculi l'IMC ingressat per l'usuari/ària, és a dir, qui ho executi haurà d'ingressar aquestes dades.
- La funció ha de classificar el resultat en les seves respectives categories.

```

def obtener_datos():
    peso = float(input("Introduzca su peso en kg: "))
    altura = float(input("Introduzca su altura en centimetros: "))

    return peso, altura

# Creo otra función para calcular el IMC y clasificarlo:

def calcular_IMC(peso, altura):
    IMC = peso / ((altura*0.01)** 2)

    if IMC <= 18.5:
        clasificacion = "BAJOPESO"
    elif 18.5 < IMC <= 24.9:
        clasificacion = "NORMOPESO"
    elif 25 <= IMC <= 29.9:
        clasificacion = "SOBREPESO"
    else:
        clasificacion = "OBESIDAD"

    return IMC, clasificacion

# Por último utilizo otra función para imprimir el resultado:

def imprimir_resultado_IMC(IMC, clasificacion):
    print(f"Tu IMC es: {IMC:.2f} y tu clasificación es {clasificacion}")

# Utilizo la función main() para organizar la ejecución del código:

def main():
    peso, altura = obtener_datos()
    IMC, clasificacion = calcular_IMC(peso, altura)
    imprimir_resultado_IMC(IMC, clasificacion)
main()

```

[1]

... Tu IMC es: 23.03 y tu clasificación es NORMOPESO

En el primer ejercicio primero creé una función llamada "obtener_datos" la cual me permite recolectar (a través de 2 inputs) los datos que necesito: altura y peso. Para poder trabajar con los datos proporcionados a

través del input, y dado que al introducir datos de esa manera python los considera texto (str) y no números, convertí ambos inputs en números decimales (float) ya que algunos de los datos pueden contener comas.

Luego creé una función llamada “calcular_IMC” que, además de hacer el calculo del IMC (una vez dados a través del input los datos del peso en kg y la altura en centímetros de la persona), clasifica el resultado obtenido en bajopeso, normopeso, sobrepeso u obesidad según los parámetros marcados por la OMS.

Por último, utilizando una tercera función, imprimo un mensaje con el resultado del calculo del IMC y la clasificación; y gracias a la función main() organizo la ejecución del código.

En el ejemplo yo introduje de peso: 65kg y de altura 168cm, y acoté el resultado del cálculo del IMC a 2 decimales utilizando {IMC:.2f}.

Exercici 2

Convertidor de temperatures:

- Existeixen diverses unitats de temperatura utilitzades en diferents contextos i regions. Les més comunes són Celsius (°C), Fahrenheit (°F) i Kelvin (K). També existeixen altres unitats com Rankine (°Ra) i Réaumur (°Re). Selecciona almenys 2 conversors, de tal manera que en introduir una temperatura retorni, com a mínim, dues conversions.

```
# Creo una función para obtener la temperatura:

def obtener_temperatura():
    temperatura = input("Escribe la unidad en la que está tu temperatura: C (para grados celsius), F (para fahrenheit) o K (para kelvin)").strip().lower()
    if temperatura == 'c':
        valor = float(input("Introduzca la temperatura en grados celsius (solo el número): "))
    elif temperatura == 'f':
        valor = float(input("Introduzca la temperatura en grados fahrenheit (solo el número): "))
    elif temperatura == 'k':
        valor = float(input("Introduzca la temperatura en kelvins (solo el número): "))
    else:
        print("Unidad no válida!")
        return None, None # retorna None si la unidad no es válida

    return valor, temperatura
```

En este segundo ejercicio primero creé la función “obtener_temperatura” en la cual utilizando un primer input hago al usuario escoger la temperatura en la que introducirá su valor de temperatura: celsius, fahrenheit o kelvin, a través de la primera letra de cada una de las 3 temperaturas; y luego convierto esta letra en minúscula y elimino espacios para evitar errores debidos a diferentes formas de escribir lo mismo. Posteriormente, utilizando otros inputs, hago que el usuario añada la temperatura y convierto el número en formato string que me devuelve en un float para poder realizar calculos matemáticos con él. También añadido un mensaje de error por si se introduce algun valor inválido.

Luego creé una función llamada “convertidor_temperatura” que, a través de cálculos matemáticos, y utilizando diferentes condicionales, convierte el valor de la temperatura obtenido a través de la función anterior (“obtener_temperatura”), dependiendo de su unidad, en las otras 4 unidades de temperatura sugeridas en el enunciado (°C, °F, K, °Ra o °Re).

Al final de todo ejecutamos el código y se imprimirá un mensaje con el valor y la temperatura introducida seguida de la conversión en las otras cuatro unidades de temperatura.

```

# Creo otra función para calcular la temperatura:

def convertidor_temperatura(valor, temperatura):
    if temperatura == 'c':
        fahrenheit = (valor * 9/5) + 32
        kelvin = valor + 273.15
        rankine = (valor + 273.15) * 9/5
        reamur = valor * 0.8
        print(f"{valor}°C equivalen a {fahrenheit:.2f}°F, {kelvin:.2f}K, {rankine:.2f}°Ra y {reamur:.2f}°Re")

    elif temperatura == 'f':
        celsius = (valor - 32) * 5/9
        kelvin = (valor + 459.67) * 5/9
        rankine = valor + 459.67
        reamur = (valor - 32) * 4/9
        print(f"{valor}°F equivalen a {celsius:.2f}°C, {kelvin:.2f}K, {rankine:.2f}°Ra y {reamur:.2f}°Re")

    elif temperatura == 'k':
        celsius = valor - 273.15
        fahrenheit = (valor * 9/5) - 459.67
        rankine = valor * 9/5
        reamur = (valor - 273.15) / 1.25
        print(f"{valor}K equivalen a {celsius:.2f}°C, {fahrenheit:.2f}°F, {rankine:.2f}°Ra y {reamur:.2f}°Re")

# Ejecutar el código
valor, temperatura = obtener_temperatura()
if valor is not None and temperatura is not None:
    convertidor_temperatura(valor, temperatura)

[1]
... 18.0°C equivalen a 64.40°F, 291.15K, 524.07°Ra y 14.40°Re

```

En este ejemplo yo introduje 18°C y también acoté el resultado de las diferentes temperaturas a 2 decimales utilizando por ejemplo {fahrenheit:.2f} en el caso de los grados fahrenheit.

Exercici 3

Comptador de paraules d'un text:

- Escriu una funció que donat un text, mostri les vegades que apareix cada paraula.

```

import re # importo el módulo re primero para poder utilizar la funcion re.sub().
# re es la librería estándar para trabajar con expresiones regulares (regex): permite buscar, reemplazar, dividir y manipular cadenas de texto de manera avanzada.

# Creo una función para contar las palabras de un texto

def contador_palabras(texto):
    texto = texto.lower() # convertimos el texto a minúsculas
    texto = re.sub(r'[\.\,\s]', '', texto) # eliminamos los signos de puntuación

    palabras = {}
    lista_palabras = texto.split() # dividimos el texto en palabras

    for palabra in lista_palabras:
        palabras[palabra] = palabras.get(palabra, 0) + 1 # usamos .get() para acceder de forma segura a los valores del diccionario sin generar errores si la clave no existe

    palabras = dict(sorted(palabras.items())) # con sorted ordeno el diccionario en orden alfabético de la key

    return palabras

# Creo una función para ejecutar la función anterior:

def ejecutar_contador_palabras():
    texto = input("Introduzca un texto: ")
    resultado = contador_palabras(texto)
    print(resultado)

# Llamo a la función 'ejecutar' para ejecutarlo todo:

ejecutar_contador_palabras()

... {'es': 1, 'la': 1, 'los': 1, 'que': 1, 'son': 1, 'sueño': 1, 'sueños': 2, 'toda': 1, 'vida': 1, 'y': 1}

```

En el tercer ejercicio, antes de crear mi función, importé “re” una librería que nos permite trabajar con expresiones regulares: buscando, reemplazando, dividiendo y manipulando cadenas de texto de manera avanzada, para luego poder utilizar `re.sub()` que nos permite eliminar todos los signos de puntuación de nuestro texto.

Luego creé la función “`contador_palabras`”, la cuál dado un texto (obtenido a través del input) primero lo convierte todo en minúsculas con `.lower`, luego elimina los signos de puntuación (con `re.sub`, como ya había comentado antes) y luego con `.split()` lo dividimos en las diferentes palabras que lo componen, creando también un diccionario vacío llamado “`palabras`”. Por último, utilizando un bucle para recorrer el texto dividido en palabras y la función `.get()` para evitar errores en caso de que no exista la palabra aun en el diccionario vacío, rellenamos el mismo haciendo un conteo de las repeticiones de cada palabra de nuestro texto: si la palabra no estaba antes en el diccionario se añade como key con un value de 1, y si la palabra ya había sido añadida antes se aumenta en 1 el value. Además, utilizando la función `sorted()` ordeno el diccionario alfabéticamente.

Creo otra función “`ejecutar_contador_palabras`” para ejecutar la función anterior y utilizando un input, introducir un texto e imprimir el diccionario del conteo de palabras al final.

En el ejemplo yo utilicé la frase: “Que toda la vida es sueño y los sueños, sueños son...”.

Exercici 4

Diccionari invers:

- Resulta que el client té una enquesta molt antiga que s'emmagatzema en un diccionari i els resultats els necessita al revés, és a dir, intercanviats les claus i els valors. Els valors i claus en el diccionari original són únics; si aquest no és el cas, la funció hauria d'imprimir un missatge d'avertiment:

```
reverse_dictionary({'a': 1, 'b': 2, 'c': 3})

{1: 'a', 2: 'b', 3: 'c'}
```

```
reverse_dictionary({'x': 'apple', 'y': 'banana', 'z': 'banana'})

Error: multiple keys for one value
```

En este ejercicio, creé una función llamada “`diccionario_inverso`” en la cual añadí un “`diccionario_nuevo`” vacío para almacenar nuestro diccionario inverso y luego utilicé un condicional dentro de un bucle `for` que itera para cada `key` y `value` de nuestro diccionario (utilizando la función `.items()`) para forzar la obtención de un mensaje de error. Normalmente si un diccionario tiene dos `keys` iguales elimina una sin darnos un error por eso se tiene que forzar el mismo y para ello yo utilicé ‘`raise`’ (dentro del condicional) que se utiliza para lanzar una excepción intencionalmente, seguido de ‘`ValueError`’ y el mensaje de error que debía salir si el `diccionario_nuevo` tenía valores repetidos. Por último, si el `diccionario_nuevo` no tenía valores repetidos simplemente se cambiaba el orden de `key` y `value` para obtener nuestro diccionario inverso.

```
def diccionario_inverso(diccionario):
    diccionario_nuevo = {}

    for key, value in diccionario.items():
        if value in diccionario_nuevo:
            raise ValueError(f"Error: múltiples keys para el value '{value}'") # lanza un error
        diccionario_nuevo[value] = key # cambia el orden clave:valor

    return diccionario_nuevo

diccionario = {'x': 'apple', 'y': 'banana', 'z': 'banana'}

# probamos que imprime error en valores duplicados:
try: # usamos try-except para que nos muestre el error si hay valores duplicados
    diccionario_nuevo = diccionario_inverso(diccionario)
    print(diccionario_nuevo)
except ValueError as e:
    print(e)
```

[1] ✓ 0.0s

... Error: múltiples keys para el value 'banana'

Primero introduje un diccionario con un value repetido (banana) y en este caso se puede observar que me sale el error notificando que está repetido.

```
diccionario2 = {'x': 'apple', 'y': 'banana', 'z': 'strawberry'}

# probamos que funciona si no hay valores duplicados:
try:
    diccionario_nuevo = diccionario_inverso(diccionario2)
    print(diccionario_nuevo)
except ValueError as e:
    print(e)
```

[2] ✓ 0.0s

... {'apple': 'x', 'banana': 'y', 'strawberry': 'z'}

Luego lo probé sustituyendo el value repetido y entonces si obtuve mi diccionario invertido.

En este ejercicio para obtener el diccionario_nuevo al final en vez de utilizar print() solamente como en los anteriores, utilicé try-except para que mostrara el error si había valores duplicados.

Nivell 2

Exercici 1

Diccionari invers amb duplicats:

- Continuant amb l'exercici 4 del nivell 1: al client es va oblidar de comentar un detall i resulta que els valors en el diccionari original poden duplicar-se i més, per la qual cosa les claus intercanviades poden tenir duplicats. En aquest cas, en l'exercici anterior imprimies un missatge d'avertiment, ara, els valors del diccionari resultant hauran d'emmagatzemar-se com una llista. Tingues en compte que si és un valor únic no ha de ser una llista.

```
reverse_dictionary_plus({'x': 'apple', 'y': 'banana', 'z': 'banana'})

{'apple': 'x', 'banana': ['y', 'z']}
```

Este ejercicio es una modificación del anterior pero en este caso lo que queremos es que en lugar de darnos un error cuando se encuentre con un value duplicado, al hacer la inversión le asigne a dicho value (que ahora es una key) las key que antes lo tenían como value pero en forma de lista dentro del diccionario. Pero todo esto será si hay duplicados, si el valor es único se almacena dentro del diccionario en forma de key:value normal.

```
# Creo una nueva función para modificar la función diccionario_inverso y almacenar los valores duplicados en una lista

def valores_duplicados(diccionario): # llama a diccionario_inverso y convierte los valores duplicados en lista
    try:
        return diccionario_inverso(diccionario) # intenta invertir el diccionario normalmente
    except ValueError:
        diccionario_nuevo = {} # creo un nuevo diccionario para manejar duplicados

        for key, value in diccionario.items():
            if value in diccionario_nuevo: # si el value ya existe, lo convertimos en lista y agregamos la nueva key
                if isinstance(diccionario_nuevo[value], list): # la función isinstance(objeto, tipo) verifica si un objeto es de un determinado tipo
                    diccionario_nuevo[value].append(key)
                else:
                    diccionario_nuevo[value] = [diccionario_nuevo[value], key]
            else:
                diccionario_nuevo[value] = key # si no existe, lo guardamos como string

        return diccionario_nuevo

diccionario = {'x': 'apple', 'y': 'banana', 'z': 'banana'}

diccionario_nuevo = valores_duplicados(diccionario)
print(diccionario_nuevo)

... {'apple': 'x', 'banana': ['y', 'z']}
```

Para conseguir estos cambios utilicé el código del ejercicio anterior y luego añadí algunas modificaciones utilizando la nueva función “valores_duplicados”, que llama a la función creada en el ejercicio 4 del nivel 1:

- Primero, utilizando un try-except intento ejecutar la función “diccionario inverso” del ejercicio anterior; pero en caso de error creamos un nuevo diccionario para almacenar los duplicados en forma de lista y para ello utilizamos un bucle que recorra nuestra lista inicial y unos condicionales dentro de dicho bucle.
- En la primera condicional del bucle for ahora en lugar de imprimirse el error lo que pasa es que lee si el valor ha sido introducido ya en el diccionario_nuevo (que está vacío en un principio) y si no lo ha estado simplemente introduce la key:value cambiándolo el orden inicial. En caso de que ya esté introducido el valor con anterioridad:
 1. Si ya existe una lista de valores para esa key, añade la ahora value a la lista de valores que comparten la misma key (para esto utilicé la función isinstance(objeto, tipo) que se encarga de verificar si un objeto es de un determinado tipo, en este caso si es una lista).
 2. Si solo hay un value anterior para esa key crea una lista de valores.

(Estas 2 posibilidades se dan a la vez que se cambia el orden key:value)

Al final, imprime el diccionario_nuevo y como se puede observar, ya que introduje un value repetido para varias keys en el diccionario original, en el diccionario_nuevo se creó una lista para almacenar las keys de dicho value al cambiar el orden key:value.

Exercici 2

Conversió de tipus de dades:

- El client rep una llista de dades i necessita generar dues llistes, la primera on estaran tots els elements que es van poder convertir en flotants i l'altra on estan els elements que no es van poder convertir. Exemple de la llista que rep el client: ['1.3', 'one' , '1e10' , 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4']]

```
conversion([ '1.3', 'one' , '1e10' , 'seven', '3-1/2', ('2',1,1.4,'not-a-number'), [1,2,'3','3.4']])

([1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4],
 ['one', 'seven', '3-1/2', 'not-a-number'])
```

Para poder llevar a cabo este ejercicio tuve que crear una función dentro de otra, la primera para procesar la lista y la segunda para aplanarla (ya que dentro de la lista inicial había otra lista y una tupla) y organizarla en dos listas finales.

Para procesar la lista creé la función “procesar_lista” en la que cree las dos listas vacías que luego utilizaré para organizar el resultado y también dentro de esta función creé la función “aplanar_y_organizar” en la cual, usando primero un condicional para clasificar si el elemento dentro de la lista es otra lista o una tupla, uso un bucle for que recorre todos los elementos de esa lista (o tupla) dentro de la lista y los procesamos

recursivamente aplanando la lista (extrayendo los elementos de dentro de la lista o tupla dentro de la lista), utilizando la función `isinstance()`.

En caso de no tratarse de una tupla o una lista, añadí un `try_except` para evitar que el programa fallara con valores no numéricos y luego intenté convertir cada elemento en un float (porque muchos de los elementos de esta lista son string, están entre comillas). Si el elemento se convertía en float se añadía, usando la función `.append()`, a la lista `flotantes`; si el intento de conversión a float daba error se añadía el elemento, usando la función `.append()`, a la lista `otros`.

Luego utilizamos otro bucle `for` para repetir el proceso anterior a cada elemento de la lista principal.

Por ultimo se imprimen ambas listas con su nombre cada una para comprobar que el resultado es correcto.

```
# Creo una función que aplanar la lista (eliminando las listas y tuplas en su interior) y clasifica los elementos en números decimales u otros

def procesar_lista(lista):
    lista_flotantes = []
    lista_otros = []

    def aplanar_y_organizar(elemento): # aplanar la lista y clasifica los elementos
        if isinstance(elemento, (list, tuple)): # la función isinstance(objeto, tipo) verifica si un objeto es de un determinado tipo
            for sub_elemento in elemento:
                aplanar_y_organizar(sub_elemento) # si es una lista o tupla, la procesamos recursivamente
        else:
            try: # usamos try-except para evitar que el programa falle con valores no numéricos
                lista_flotantes.append(float(elemento)) # intentamos convertir a float, si funciona: es un número decimal (flotante)
            except (ValueError, TypeError): # valor incorrecto o tipo de dato incorrecto para la operación
                lista_otros.append(elemento) # si no se puede convertir, lo agregamos a otra lista

    for elemento in lista: # procesamos la lista principal
        aplanar_y_organizar(elemento)

    return lista_flotantes, lista_otros

lista = ['1.3', 'one', '1e10', 'seven', '3-1/2', ('2', 1, 1.4, 'not-a-number'), [1, 2, '3', '3.4']]

lista_flotantes, lista_otros = procesar_lista(lista) # llamar a la función

print(f"lista_flotantes: {lista_flotantes}")
print(f"lista_otros: {lista_otros}")
```

[7]

```
... lista_flotantes: [1.3, 10000000000.0, 2.0, 1.0, 1.4, 1.0, 2.0, 3.0, 3.4]
lista_otros: ['one', 'seven', '3-1/2', 'not-a-number']
```


Nivell 3

Exercici 1

Comptador i endreçador de paraules d'un text:

- El client va quedar content amb el comptador de paraules, però ara vol llegir arxius TXT i que calculi la freqüència de cada paraula ordenades dins de les entrades habituals del diccionari segons la lletra amb la qual comencen, és a dir, les claus han d'anar de la A a la Z i dins de la A hem d'anar de la A la Z. Per exemple, per a l'arxiu "tu_me_quieres_blanca.txt" la sortida esperada seria:

```
{ 'a': { 'a': 3,
  'agua': 1,
  'al': 2,
  'alba': 4,
  'alcobas': 1,
  'alimenta': 1,
  'alma': 1,
  'amarga': 1,
  'azucena': 1},
  'b': { 'baco': 1,
  'banquete': 1,
  'bebe': 1,
  'blanca': 3,
  'boca': 1,
  'bosques': 1,
  'buen': 1},
  'c': { 'cabañas': 1,
  'carnes': 2,
  'casta': 3,
  'cerrada': 1,
  'con': 4,
  'conservas': 1,
  'copas': 1,
  'corola': 1,
  'corriste': 1,
  ...
  'tornadas': 1,
  'tú': 8},
  'u': { 'un': 1, 'una': 1},
  'v': { 'vestido': 1, 'vete': 1, 'vive': 1},
  'y': { 'y': 5}}
```

Para resolver este ejercicio creé una nueva función llamada “contador_texto_ordenado”, la cuál lo primero que hace es abrir y leer el documento .txt en el cual se encuentra almacenado el texto, para eso utilizamos with open() para abrir el documento, .read() posteriormente para leerlo y .replace() para remplazar los saltos de linea (ya que el texto es un poema) por espacios entre palabras.

Una vez leído el texto pasa a su procesamiento utilizando la función “contador_palabras()” que creé en el ejercicio 3 del nivel 1 y almacena el resultado en una lista llamada ‘palabras_contadas’; luego crea un diccionario vacío “dicc_texto” en el que almacenaremos el resultado final.

Después, utilizando un bucle for, recorre la lista ‘palabras_contadas’ y va obteniendo la primera letra de cada una de las palabras que la componen, y utilizando un condicional:

- Si la letra inicial no ha sido añadida anteriormente a 'dicc_texto': la añadimos creando un diccionario vacío a su vez dentro de este con la letra inicial como key del mismo.
- Si la letra inicial ya está en 'dicc_texto': añadimos la palabra que empieza por esa letra desde la lista 'palabras_contadas', así como la cantidad de veces que está repetida dicha palabra en el texto.

```
# Creo una nueva función que llama a la función 'contador_palabras()' y luego crea un diccionario con las palabras contadas

def contador_texto_ordenado(nombre_archivo):
    with open(nombre_archivo, "r", encoding="utf-8") as archivo:
        texto = archivo.read().replace("\n", " ") # leemos el contenido del archivo y eliminamos los saltos de línea

    palabras_contadas = contador_palabras(texto) # llamamos a la función 'contador_palabras()' para contar las palabras de nuestro texto

    dicc_texto = {}
    for palabra, cantidad in palabras_contadas.items(): # organizamos por la letra inicial de cada palabra
        letra_inicial = palabra[0] # obtenemos la primera letra

        if letra_inicial not in dicc_texto:
            dicc_texto[letra_inicial] = {} # creamos la key si no existe

        dicc_texto[letra_inicial][palabra] = cantidad # asignamos la cantidad

    return dicc_texto

ruta_archivo = "/Users/liss/Desktop/ItAcademy/2-Especialización/iSprints!/S7/Datos/tu_me_quieres_blanca.txt" # ruta del archivo de texto

dicc_texto_ordenado = contador_texto_ordenado(ruta_archivo) # llamamos a la función con el nombre del archivo

dicc_texto_ordenado # visualizamos el resultado

[3] ✓ 0.0s

... {'a': {'a': 3,
      'agua': 1,
      'al': 2,
      'alba': 4,
      'alcobas': 1,
      'alimenta': 1,
      'alma': 1,
      'amarga': 1,
```

Por ultimo, para poder analizar nuestro texto añadimos la ruta en la que se encuentra el archivo y visualizamos el diccionario 'dicc_texto_ordenado' que contendrá todas las palabras del texto ordenadas dentro de otro diccionario que se encuentra dentro del anterior cuyo nombre se corresponde con la letra inicial de las palabras (ordenadas alfabeticamente como en un diccionario de la RAE) y dentro de cada uno de estos diccionarios dentro de 'dicc_texto_ordenado' tendremos las palabras con su conteo de aparición en formato key:value como se puede observar en el fragmento del resultado de la imagen.